

소마 메이트(SoMa Mate) 프로젝트 기획서

소프트웨어 마에스트로 연수생을 위한 에이전트형 멘토·멘토링 탐색 그룹 확장 프로그램

참여자 : 이성현, 김민수, 김수민, 한우진, 송수민

1. 문제 정의 및 프로젝트 개요

프로젝트 한 줄 정의 [필수]	연수생이 자연어로 질문하면, 소마 포털에 흩어져 있는 멘토·멘토링·특강·팀매칭 정보를 탐색·필터·정리해 답해주는 에이전트형 그룹 확장 프로그램을 만드는 팀이다.
서비스 한 줄 정의 (사용자 관점) [필수]	“내 기술 스택과 목표(취업/창업)에 맞는 멘토와 멘토링·특강을 알아서 찾아 정리해주는 소마 전용 AI 비서.”
서비스 선정 배경 [권장]	소마 연수생에게 필요한 정보(멘토 프로필, 멘토링 접수 현황, 특강 일정, 팀매칭 현황)는 여러 페이지에 분산되어 있어, 원하는 조건의 정보를 찾으려면 페이지를 일일이 열고 표를 직접 훑어야 한다. “Spring을 잘 아는 멘토가 누구인지”, “창업에 도움 줄 멘토가 누구인지” 같은 맥락형 질문은 기존 UI로는 한 번에 답을 얻기 어렵다. 자연어로 묻고 맥락에 맞춰 답을 받는 경험이 필요하다고 판단했다.
해결하려는 문제 [필수]	① 정보 분산으로 인한 높은 탐색 비용 – 멘토/멘토링/특강/팀매칭 정보가 분리되어 있어, 한 가지 의사결정에 여러 페이지를 오가야 한다. ② 맥락 기반 탐색의 부재 – “내 스택·목표에 맞는” 같은 조건 필터가 약해, 연수생이 직접 눈으로 매칭해야 한다. 문제라고 본 이유 – 연수 기간은 짧고 결정(멘토 신청·팀 합류)은 빠르게 내려야 하는데, 정보 탐색에 드는 시간이 의사결정 품질을 떨어뜨린다.
대상 사용자 [필수]	소프트웨어 마에스트로 연수생. 특히 (1) 특정 기술 스택을 보유하고 그에 맞는 멘토를 찾는 연수생, (2) 취업 또는 창업이라는 목표가 뚜렷한

	연수생, (3) 관심 분야의 멘토링·특강을 빠르게 파악하고 싶은 연수생, (4) 팀 빌딩 현황을 확인하려는 연수생.
핵심 가치 [필수]	흩어진 소마 정보를, 연수생 개인의 맥락(보유 스택·목표·관심 분야)에 맞춰, 자연어 질문 한 번으로 정리해 주는 것. ‘검색’이 아니라 ‘대신 찾아주고 정리해주는’ 경험에 집중한다.

2. 사용자 및 Agent 설계

타겟 사용자 페르소나 [권장]	예시 페르소나 – 김연수 (26세) - 백엔드 지망 연수생. 보유 스택: Java/Spring, AWS 기초. - 목표: 취업(서버 개발자). 관심 분야: 대규모 트래픽 처리, 클라우드. - 특징: 시간이 부족하고, 여러 페이지를 뒤지는 정보 탐색을 번거로워함. 결정을 빨리 내리고 싶어 함.
Agent의 역할 [필수]	연수생의 요구(스택·목표·관심 분야·질문)를 해석하고, 소마 데이터에서 적절한 정보를 검색·필터·교차참조한 뒤, 자연어로 요약·추천하는 정보 탐색 에이전트. 구체적으로 수행하는 일 - notion 크롤링을 통해 초기 데이터를 취합 - 자연어를 입력으로 받아 질문을 분류하고 해당 작업을 스스로 판단하여 수행하고, 개별 답변을 취합함. - 조건에 맞는 멘토 리스트업 (예: ‘Spring + 창업 멘토링 경험’ 멘토 추리기). - 팀매칭 현황 요약 (모집 중 팀, 빈 포지션 등). - 여러 정보를 합쳐야 하는 질문에 대해 단계적으로 도구를 호출해 종합 답변 생성.
Agent의 성격 및 톤앤매너 [권장]	간결하고 실무적이되, 먼저 연수를 경험한 ‘선배’ 같은 친근한 톤. 추측이나 과장 없이 근거(출처 항목)를 함께 제시하고, 정보가 없으면 없다고 명확히 말한다.
Agent의 자율성 범위 [필수]	스스로 수행 (자율)

	– 질문 의도 파악, 어떤 데이터·도구를 쓸지 판단, 검색·필터·교차참조, 결과 요약·추천까지. 사람 확인 후 수행 (반드시 멈춤) – 멘토링 신청, 팀 합류 등 ‘되돌릴 수 없는 실제 액션’은 직접 실행하지 않고, 해당 신청 페이지 링크 안내까지만 수행한다. – 개인정보·로그인 자격 증명 입력은 사용자가 직접 한다. 이렇게 ‘읽기·정리는 자율, 쓰기·신청은 안내까지’로 경계를 뒤 안전성과 구현 난이도를 함께 조절한다.
--	---

3. 핵심 기능 및 사용자 흐름

주요 사용자 시나리오 [필수]	시나리오 A – 멘토 찾기 – “React랑 Node 잘 아는, 창업 경험 있는 멘토 추천해줘” → 조건에 맞는 멘토 여러 명을 이유와 함께 리스팅업 -> 소마 메이트가 조건에 맞는 적절한 멘토를 선정 -> 사용자에게 적절한 멘토를 추천. 시나리오 B – 멘토링/특강 정리 – “클라우드 쪽 지금 신청할 수 있는 거 뭐 있어?” → 접수 중인 클라우드 관련 멘토링·특강을 리스팅업 -> 사용자의 기존 멘토링 일정과 겹치지 않는 멘토링, 특강 선정 -> 사용자에게 적절한 멘토링, 특강 추천. 시나리오 C – 팀매칭 현황 – “프론트 구하는 팀 있어?” → 연수생 중 프론트 관련 분야를 희망하는 사람들을 찾는다 -> 이미 팀이 있는지 여부를 확인.
핵심 기능 정의 [필수]	① 자연어 질의 이해 — 연수생의 문장에서 스택·목표·관심분야·요청 유형(의도)을 추출. ② 멘토 매칭 — 스택/목표/멘토링 이력 기준으로 멘토를 필터·정렬해 추천. ③ 멘토링·특강 정리 — 관심 분야 입력 시 접수 중 항목을 모아 마감일·정원과 함께 요약. ④ 팀매칭 현황 안내 — 모집 중 팀과 빈 포지션 요약. (MVP 이후 확장)

	⑤ 종합 응답 생성 — 위 도구의 결과를 합쳐 출처와 함께 자연어로 답변.
사용자 관점 워크플로우 [필수]	1. 소마 포털에서 확장 프로그램 아이콘 클릭 → 채팅 패널 열림. 2. 자연어로 질문 입력 (예: 보유 스택·목표를 함께 언급). 3. Agent가 질문 의도와 필요한 정보를 안내하며 진행. 4. 정리된 결과(멘토/멘토링/팀 목록)를 카드 형태로 확인. 5. 신청이 필요하면 Agent가 안내한 링크로 이동해 사용자가 직접 신청.
시스템 관점 워크플로우 (Agentic Loop) [권장]	① 의도 분류(Intent) — LLM이 질문을 분석해 필요한 작업과 파라미터(스택·분야 등) 도출. ② 계획·도구 선택(Plan) — 멘토검색 / 멘토링·특강조회 / 팀매칭조회 중 호출할 도구(Tool)를 결정. ③ 실행(Act) — 선택한 도구를 호출해 데이터 검색·필터. 한 번에 안 되면 결과를 보고 다음 도구를 추가 호출(다단계 추론). ④ 종합(Synthesize) — 수집한 결과를 근거와 함께 자연어 답변으로 정리. ⑤ 검증·반복(Reflect) — 결과가 불충분하면 추가 질의나 재검색을 수행하고, 충분하면 응답 종료.

4. 기술 구현 설계

기술 스택 [필수]	- 프론트엔드 / 확장: Chrome Extension (Manifest V3), React + TypeScript, 사이드패널/팝업 UI. - 에이전트 오케스트레이션: LLM API(예: Claude / GPT) + 함수 호출(Tool Calling) 기반 에이전트 루프 (LangChain 또는 직접 구현). - 백엔드: Node.js(Express) 또는 Python(FastAPI) — 인증 검증 및 데이터 제공 ‘문지기’ 서버. - 데이터: 소마 정보(멘토/멘토링/특강/팀)를 적재한 자체 DB(PostgreSQL 등) + 의미 검색용 임베딩/벡터 인덱스(RAG).
---------------------------------	--

시스템 아키텍처 [권장]	• [크롬 확장 UI] → 사용자 질문 전달. • [에이전트 코어] 의도 분석 → 도구 선택 → 호출 → 결과 종합. • [Tools] 멘토검색 · 멘토링/특강조회 · 팀매칭조회 (각각 백엔드 API 호출). • [백엔드/문지기 서버] 요청자의 로그인·소마 회원 여부를 검증한 뒤에만 데이터 응답. • [데이터 저장소] 자체 DB + 벡터 인덱스. 민감 데이터는 확장 프로그램에 내장하지 않는다.
프롬프트 설계 전략 [권장]	• 역할 고정: ‘소마 연수생 전용 정보 탐색 도우미’로 시스템 프롬프트에 역할·범위를 명시. • 입출력 통제: 어떤 도구를 언제 호출할지 규칙화하고, 답변은 ‘근거(출처 항목) + 요약’ 형식으로 강제. • 환각 방지: 검색 결과에 없는 멘토·일정은 만들어내지 말고 ‘정보 없음’으로 응답하도록 지시.
데이터 활용 및 기억 관리 [권장]	• 세션 기억: 대화 중 언급된 사용자의 스택·목표·관심 분야를 세션 동안 유지해 재질문 시 반복 입력을 줄인다. • 데이터 갱신: 멘토링·팀매칭처럼 자주 바뀌는 정보는 주기적으로 동기화하여 최신성을 유지. • RAG: 멘토 프로필·특강 설명 등 텍스트는 임베딩해 의미 기반 검색으로 매칭 정확도를 높인다.
제약사항 및 예외 처리 [필수]	접근 제어 (핵심 제약) – 소마 관계자가 아닌 사람의 데이터 열람을 막아야 한다. 소마 서버 자체는 수정 불가하므로, 자체 백엔드가 로그인·회원 여부를 검증한 요청에만 데이터를 응답한다(데이터는 확장 프로그램에 내장하지 않음). 이번 범위에서 제외(Out of Scope) – 멘토링 신청·팀 합류 등 실제 쓰기 액션의 자동 실행은 제외(안내까지만). – 개인정보·결제·로그인 자격 증명의 자동 입력은 제외. 예외 처리

	- 검색 결과 없음, 데이터 동기화 실패, 미로그인 상태는 사용자에게 명확한 안내 메시지로 처리. 개인정보 유의 - 멘토·연수생의 개인정보를 다루므로 수집·보관 범위와 소마 운영 정책 부합 여부를 사전 확인.
--	---

5. 성과 평가 및 실행 계획

성공 지표 (KPI) [권장]	• “이 정도면 성공” 기준: 연수생이 원하는 멘토/멘토링/팀 정보를 1~2회 질문 안에 정확히 받을 수 있다. • 정성: 데모 사용자가 ‘직접 찾는 것보다 빠르고 편하다’고 응답. • 정량(가능 시): 주요 질의 유형의 응답 정확도, 평균 응답 소요 시간.
MVP 범위 [필수]	로컬 환경에서 실행 가능한 Agentic Workflow 데모를 목표로 한다. 반드시 만든다 - 자연어 질문 → 의도 분석 → 분기 처리 → 데이터 검색 → 종합 응답으로 이어지는 에이전트 루프 1개 완성. - 핵심 기능 중 ‘멘토 리스트업’과 ‘멘토링·특강 정리’ 2가지를 실제 동작하게 구현(샘플/모의 데이터 허용). - 크롬 확장 UI에서 질문하고 결과를 보는 기본 흐름. 이번엔 하지 않는다 (다음 단계로) - 팀매칭 현황 안내 기능 — 핵심 기능으로 계획하되, MVP 이후 확장 단계에서 구현. - 실서비스 수준의 인증 연동 완성, 자동 신청, 대규모 실데이터 연동.
단계별 개발 로드맵 [필수]	• 1단계: 데이터 스키마 정의 + 샘플 데이터 적재(멘토/멘토링). • 2단계: 도구(API) 구현 — 멘토검색, 멘토링/특강조회 (MVP 2개). • 3단계: 에이전트 루프 구현 — 의도 분석 → 분기 처리 → 종합 응답. • 4단계: 크롬 확장 UI 연결(사이드패널 채팅). • 5단계: 접근 제어(로그인 검증) 설계 적용 + 예외 처리 보강.

	● 6단계: 데모 시연 및 피드백 반영. ● (이후 확장) 팀매칭조회 도구 추가 — 팀/포지션 데이터 적재 후 에이전트에 연결.
기대 효과 [권장]	● 연수생이 정보 탐색에 쓰던 시간을 줄여 멘토 선택·팀빌딩 같은 핵심 결정에 집중할 수 있다. ● 맥락 기반 추천으로 스택·목표에 더 잘 맞는 멘토링 매칭이 이루어진다. ● 에이전트형 워크플로우의 실용 사례로, 이후 소마 내 다른 정보 도메인으로 확장할 토대가 된다.